MBLP131 · Hafta 6 · Ders Notları

İşletim Sistemleri Temelleri

Bellek Yönetimi: Bitişik Tahsis, Sayfalama ve Bölütleme

Birçok program, tek bir RAM: her süreç kendine ait, korunmuş bir bellek parçasını nasıl alır?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. İşletim sisteminin belleği neden yönetmesi gerektiğini açıklamak; derleme, yükleme ve yürütme zamanında adres bağlamayı birbirinden ayırmak.
2. Mantıksal (sanal) adresi fiziksel adresten ayırmak; taban (yer değiştirme) ve sınır yazmaçlı bir bellek yönetim biriminin (MMU) adresleri nasıl çevirip koruduğunu onluk ve onaltılık tabanda göstermek.
3. Takas sürelerini hesaplamak ve modern sistemlerin neden bütün süreçleri değil sayfaları takas ettiğini açıklamak.
4. İlk uyan, en iyi uyan ve en kötü uyan yöntemlerini bir delik listesine uygulamak; dış ve iç parçalanmayı, sıkıştırmayı ve %50 kuralını açıklamak.
5. Bir adresi sayfa numarası ve ofsete bölmek, bir sayfa tablosu üzerinden çevirmek, sayfa tablosu boyutunu ve iç parçalanmayı hesaplamak.
6. TLB'li sayfalı bir belleğin etkin erişim süresini hesaplamak.
7. Bölütlenmiş bir $\langle s, d \rangle$ adresini bölüt tablosuyla çevirmek, sayfalama ile bölütlemeyi karşılaştırmak ve x86-64 Linux'un gerçekte neyi kullandığını söylemek.
8. Bir Linux sürecinin sanal adreslerini, sayfa boyunu ve bellek haritasını gösteren küçük C programlarını çalıştırmak.

İçindekiler

1 Bellek Neden Yönetilir? 1

2 Adresler ve Bellek Yönetim Birimi 2

3 Takas 4

4 Bitişik Bellek Tahsisi 5

5 Sayfalama 8

6 Bölütleme 13

7 Uygulama: Linux'ta Gerçek Adresler 14

8 Özet 18

9 Sözlük 18

10 Kendiniz Deneyin 19

11 Tekrar Soruları 20

12 İleriye Bakış: 7. Hafta 21

1 Bellek Neden Yönetilir?

Geçen hafta (5. hafta) işletim sisteminin *işlemciyi* süreçler arasında nasıl paylaştırdığını inceledik: FCFS, SJF/SRTF, Öncelik ve Round Robin gibi zamanlama algoritmaları sıradaki süreci seçer; iş parçacıkları ortak veriyi paylaştığında mutex ve semaforlar yarış durumlarını önler. 2. haftanın diliyle bu, *zaman çoğullama* idi. Bu haftanın konusu *alan çoğullama*: **ana belleği** (RAM) aynı anda birçok süreç arasında paylaştırmak.

Önceki haftalardan iki olguyu hatırlayalım:

- **Bellek hiyerarşisi (2. hafta).** İşlemci komutları yalnızca yazmaçlardan ve ana bellekten yürütebilir, verileri yalnızca oradan yükleyebilir (önbellekler RAM'in kopyalarını tutar). Disklere ve SSD'lere yalnızca G/Ç yoluyla ulaşılır. Bu yüzden bir programın çalışabilmesi için (en azından kısmen) RAM'de olması gerekir.
- **Sürecin bellek düzeni (3. hafta).** Her süreç belleğini *metin* (kod), *veri* (global değişkenler), *öbek* (heap, `malloc`) ve *yığıt* (stack; yerel değişkenler, dönüş adresleri) olarak görür.

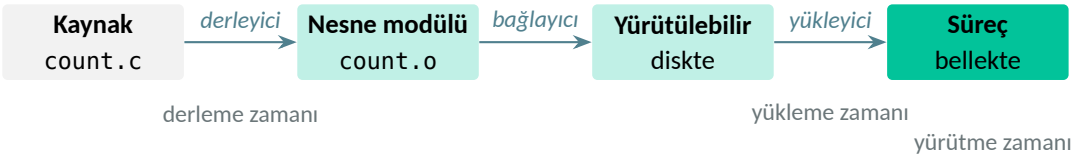
Çoklu programlamada bu süreçlerden birçoğu aynı anda RAM'de bulunur. İşletim sistemi, donanımın yardımıyla üç sorunu çözmek zorundadır:

1. **Tahsis.** RAM'in hangi kısmı hangi sürece verilecek? RAM dolarsa ne olacak?
2. **Koruma.** Bir süreç başka bir sürecin ya da çekirdeğin belleğini okuyamamalı, üzerine yazamamalıdır. Bir program belleğe saniyede yüz milyonlarca kez dokunduğundan her erişim denetlenmelidir; bu da ancak *donanımla* mümkündür.
3. **Yer değiştirme (relocation).** Program nereye yükleneceğini önceden bilmez ve işletim sistemi onu daha sonra taşımak isteyebilir.

2 Adresler ve Bellek Yönetim Birimi

2.1 Adres bağlama

Bir C programında `count = count + 1;` yazarsınız. `count` değişkeni bir *addır*; işlemci ise bir *sayıya*, yani `count`'u tutan baytların adresine ihtiyaç duyar. Adları adreslere dönüştürmeye **adres bağlama** (address binding) denir. Kaynak koddan çalışan bir sürece giden yolda program birkaç araçtan geçer ve bağlama üç farklı zamanda yapılabilir.



Şekil 1: Kaynak koddan çalışan bir sürece.

Derleme zamanı (compile time)
Sürecin bellekte nerede duracağı derleme sırasında biliniyorsa derleyici **mutlak kod** (absolute code) üretebilir. Başlangıç konumu değişirse program yeniden derlenmelidir. MS-DOS `.COM` programları böyle çalışırdı.

Yükleme zamanı (load time)
Aksi hâlde derleyici **yer değiştirebilir kod** (relocatable code) üretir: "bu modülün başından 14 bayt ötede" gibi adresler. Yükleyici, program belleğe alınırken bunları mutlak adreslere çevirir. Yüklendikten sonra program taşınamaz.

Yürütme zamanı (execution time)
Süreç çalışırken taşınabilecekse bağlama, komut gerçekten çalışana kadar ertelenmelidir: her adres, her bellek erişiminde donanım tarafından çevrilir. Bu, özel bir donanım (MMU) gerektirir; ama tüm modern genel amaçlı işletim sistemlerinin yaptığı budur.

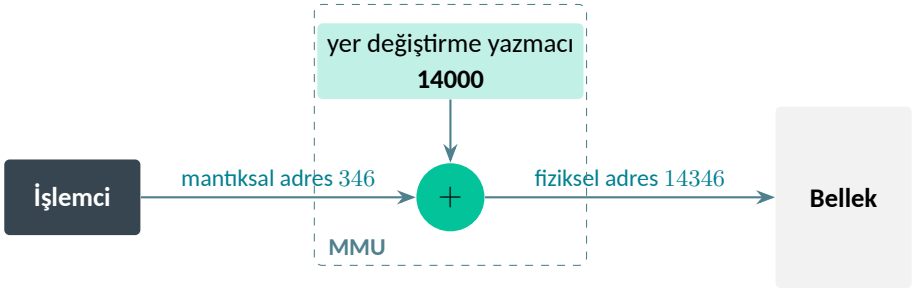
2.2 Mantıksal ve fiziksel adres

İşlemcinin bir programı yürütürken ürettiği adres **mantıksal adrestir** (logical address); yürütme zamanında bağlamada buna **sanal adres** (virtual address) da denir. Bellek biriminin gerçekte gördüğü, bellek yoluna (bus) konan adres ise **fiziksel adrestir** (physical address). Bir programın üretebileceği tüm mantıksal adreslerin kümesi onun **mantıksal adres uzayıdır**; bunlara karşılık gelen fiziksel adreslerin kümesi de **fiziksel adres uzayıdır**.

Derleme ve yükleme zamanında bağlamada mantıksal ve fiziksel adresler aynıdır. Yürütme zamanında bağlamada farklıdırlar ve çalışma anındaki çeviriyi **bellek yönetim birimi** (memory-management unit, MMU) adı verilen bir donanım aygıtı yapar. Kullanıcı programı fiziksel adresleri hiçbir zaman görmez: `printf("%p")` ile yazdırdığınız her işaretçi mantıksaldır. Bunu Bölüm 7'de deneyle kanıtlayacağız.

2.3 En basit MMU: yer değiştirme yazmacı

En basit MMU tek bir **yer değiştirme yazmacı** (relocation register) içerir; buna **taban yazmacı** (base register) da denir. Bu yazmacın değeri, bir kullanıcı sürecinin ürettiği her adrese, adres belleğe gönderildiği anda eklenir.



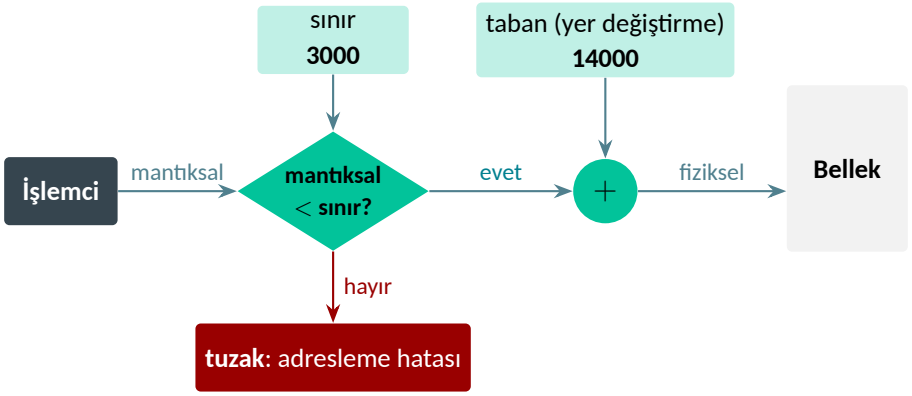
Şekil 2: Yer değiştirme yazmacıyla dinamik yer değiştirme.

Program 0 ile 2999 arasındaki adresleri kullandığını sanır. İşletim sistemi onu 14000 fiziksel adresine yüklemiş ve yer değiştirme yazmacına 14000 yazmıştır. İşlemci "346 adresindeki sözcüğü yükle" komutunu yürüttüğünde MMU belleğe $14000 + 346 = 14346$ gönderir. Süreci taşımak için işletim sistemi onun baytlarını başka bir yere kopyalar ve tek bir yazmacı değiştirir. Yer değiştirme yazmacını yüklemek *ayrıcalıklı* bir komuttur (2. hafta): onu yalnızca çekirdek değiştirebilir.

2.4 Taban ve sınır yazmaçlarıyla koruma

Yer değiştirme tek başına hiçbir şeyi korumaz: bir program çok büyük bir mantıksal adres kullanıp başka bir sürecin alanına ulaşabilir. İkinci bir yazmaç, **sınır yazmacı** (limit register), mantıksal adres uzayının boyunu tutar. MMU önce mantıksal adresi sınırla karşılaştırır, ancak ondan sonra tabanı ekler:

$0 \le a_{\text{mant}} < \text{sınır}$ ise: $a_{\text{fiz}} = \text{taban} + a_{\text{mant}}$; değilse: işletim sistemine tuzak (adresleme hatası).



Şekil 3: Taban ve sınır yazmaçlarıyla donanım düzeyinde adres koruması.

Denetim donanımda, her erişimde ve toplamayla paralel yapıldığından korumanın zaman maliyeti neredeyse sıfırdır. Denetim başarısız olursa işlemci bir *tuzak* (trap; yazılım kaynaklı bir kesme, 2. hafta) üretir ve işletim sistemi genellikle süreci sonlandırır. Linux'ta bunu *Segmentation fault (core dumped)* iletisi olarak görürsünüz; bugün bu tuzağı sayfalama donanımı üretir (Bölüm 5.6), ama fikir aynıdır.

Onluk ve onaltılık tabanda taban ve sınır

Onluk. taban $= 14000$, sınır $= 3000$.

- $a = 346$: $346 < 3000$, dolayısıyla $a_{\text{fiz}} = 14000 + 346 = 14346$.
- $a = 2999$: $2999 < 3000$, dolayısıyla $a_{\text{fiz}} = 16999$; sürecin kullanabileceği son bayt.
- $a = 3500$: $3500 \geq 3000$: **tuzak**.

Süreç tam olarak $[\text{taban},\ \text{taban} + \text{sınır}) = [14000,\ 17000)$ fiziksel aralığına sahiptir.
Onaltılık. taban $=$ `0x4000` $(= 16384)$, sınır $=$ `0x1000` $(= 4096)$.

- $a =$ `0x02A4` $(= 676)$: `0x02A4` $<$ `0x1000`, dolayısıyla $a_{\text{fiz}} =$ `0x4000` $+$ `0x02A4` $=$ `0x42A4`. Onluk tabanda kontrol: $16384 + 676 = 17060 =$ `0x42A4`.
- $a =$ `0x1200`: `0x1200` $\geq$ `0x1000`: **tuzak**.

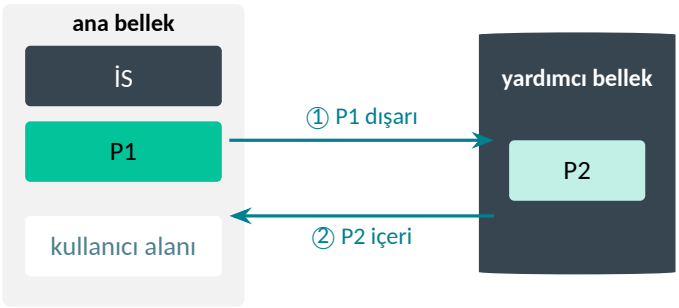
Tabanın alt üç onaltılık basamağı sıfır olduğundan toplama işlemi ofseti bu basamaklara yazmaktan ibarettir. Geçerli fiziksel aralık `0x4000–0x4FFF`'tir.

Gerçek makinelerde taban ve sınır

CDC 6600 süper bilgisayarı (1964) her işi RA (reference address) ve FL (field length) adlı iki yazmaçla koruyordu; bu, tam olarak burada anlatılan taban/sınır çiftidir. IBM System/360 farklı bir yöntem kullandı (2 KB'lık bloklar üzerinde 4 bitlik bellek anahtarları). Günümüz bilgisayarlarında taban/sınır fikri x86 işlemcilerinin bölüt tanımlayıcılarında (segment descriptor) yaşamaya devam ediyor (Bölüm 6.4).

3 Takas

Tüm süreçler için yeterli RAM yoksa işletim sistemi bir süreci geçici olarak bellekten çıkarıp bir **yardımcı belleğe** (backing store; hızlı bir disk ya da SSD) taşıyabilir ve daha sonra geri getirebilir. Buna **takas** (swapping) denir. Dışarı takas edilmiş bir süreç, yeniden içeri alınana kadar çalışamaz; yürütme zamanında bağlama kullanılıyorsa RAM'de farklı bir yere geri dönebilir.



Şekil 4: Yardımcı bellek olarak bir disk kullanılarak iki sürecin takası (İS: işletim sistemi).

Takasın maliyetine *aktarım süresi* hâkimdir ve bu süre taşınan bellek miktarıyla orantılıdır.

Bir takas ne kadar sürer?

Bir süreç 100 MB yer kaplıyor; yardımcı bellek 50 MB/s hızla aktarıyor. Süreci dışarı takas etmek

$$t_{\text{dışarı}} = \frac{100\ \text{MB}}{50\ \text{MB/s}} = 2\ \text{s}$$

sürer; aynı boyda başka bir süreci içeri almak da 2 s daha: disk gecikmesi dışında toplam yaklaşık 4 s. Bunu 10 ms'lik bir Round Robin zaman dilimiyle (5. hafta) karşılaştırın: tek bir takas $4\ \text{s} / 10\ \text{ms} = 400$ dilime eşittir.
Buna karşılık tek bir 4 KB'lık sayfayı aynı hızla taşımak $4096\ \text{B} / (50 \times 10^6\ \text{B/s}) \approx 82\ \mu\text{s}$ sürer.

Bu nedenle bütün süreçlerin standart takası, genel amaçlı sistemlerde artık en fazla acil durumlarda kullanılır. Linux, Windows ve macOS bunun yerine tek tek *sayfaları* takas eder (Bölüm 5); ayrıntılar, yani isteğe bağlı sayfalama ve sayfa değiştirme, 7. haftanın konusudur. Mobil sistemler (iOS, Android) genellikle hiç takas yapmaz: bellek azaldığında arka plandaki uygulamalardan durumlarını kaydetmelerini ister, sonra onları sonlandırır.

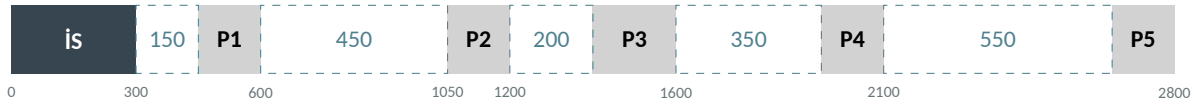
4 Bitişik Bellek Tahsisi

Bitişik bellek tahsisinde (contiguous allocation) her süreç fiziksel bellekte bitişik tek bir blok kaplar. Taban ve sınır yazmaçlarıyla bu doğal bir yöntemdir: bütün süreci tek bir sayı çifti tanımlar.

4.1 Değişken bölümler ve delik listesi

Değişken bölüm (variable partition) yönteminde işletim sistemi her sürece tam ihtiyacı kadar büyüklükte bir blok verir. Başlangıçta tüm kullanıcı belleği tek bir büyük boş bloktur. Süreçler gelip gittikçe bellek, tahsis edilmiş bloklarla **delik** (hole) adı verilen boş blokların karışımına dönüşür. İşletim sistemi deliklerin bir listesini tutar. Bir süreç geldiğinde yeterince büyük bir delik arar, gerekeni tahsis eder ve kalanı daha küçük bir delik olarak bırakır. Bir süreç sonlandığında bloğu deliğe dönüşür ve komşu deliklerle birleştirilir.

Bu bölüm boyunca tek bir örnek kullanıyoruz. Bellek 2800 KB'tır (sayılar okunaklı kalsın diye oyuncak bir boyut); ilk 300 KB'ı işletim sistemi (İS) kaplar, beş süreç P1–P5 bellektedir ve beş delik vardır:



Şekil 5: Sürekli kullanılan örnek: delik boyları KB cinsinden, adresler (KB) çubuğun altında.

Adres sırasıyla (başlangıç, boy) çiftleri olarak delik listesi şudur:

$$(300, 150),\ (600, 450),\ (1200, 200),\ (1600, 350),\ (2100, 550), \qquad \text{toplam boş} = 1700\ \text{KB}.$$

4.2 Yerleştirme algoritmaları: ilk uyan, en iyi uyan, en kötü uyan

Yeni bir süreç hangi deliği almalıdır? Üç klasik strateji şunlardır:

İlk uyan (first-fit)
Yeterince büyük *ilk* deliği tahsis et. Arama, uygun bir delik bulunur bulunmaz durabilir (listenin başından ya

da son aramanın bittiği yerden başlanarak; ikincisine sonraki uyan, next-fit, denir).

En iyi uyan (best-fit)
Yeterince büyük *en küçük* deliği tahsis et. Liste boya göre sıralı tutulmuyorsa tamamı taranmalıdır. En küçük artık deliği üretir.

En kötü uyan (worst-fit)
En büyük deliği tahsis et. Bu da tam bir tarama gerektirir. Fikir, artığın işe yarayacak kadar büyük kalmasıdır.

Dört istek, üç strateji

Süreçler şu sırayla geliyor: A $= 220$ KB, B $= 400$ KB, C $= 130$ KB, D $= 430$ KB. Her istek, seçilen deliğin başından kesilip alınır.
A isteği (220 KB). Yeterince büyük delikler: 450, 350, 550. İlk uyan adres sırasındaki ilk deliği, $450@600$'ü alır; en iyi uyan en küçüğünü, $350@1600$'ü; en kötü uyan en büyüğünü, $550@2100$'ü. Aynı biçimde devam edersek:

İstek	İlk uyan	En iyi uyan	En kötü uyan
A = 220	$450@600 \to$ kalan 230	$350@1600 \to$ kalan 130	$550@2100 \to$ kalan 330
B = 400	$550@2100 \to$ kalan 150	$450@600 \to$ kalan 50	$450@600 \to$ kalan 50
C = 130	$150@300 \to$ kalan 20	$130@1820 \to$ kalan 0	$350@1600 \to$ kalan 220
D = 430	beklemek zorunda	$550@2100 \to$ kalan 120	beklemek zorunda

İlk uyanda D geldiğinde 950 KB boştur, ama en büyük delik yalnızca 350 KB'tır; en kötü uyanda en büyük delik 330 KB'tır. Bu örnekte dört sürecin hepsini yalnızca en iyi uyan yerleştirebiliyor.

Tablo, küçük bir Python betiğiyle, `code/fit.py`, doğrulandı. Betiğin kalbi deliği seçen fonksiyondur; geri kalanı istekler üzerinde döner ve sonucu yazdırır (program çıktısı İngilizcedir: `hole` = delik, `leftover` = kalan, `waits` = bekler):

```
def choose(holes, size, rule):
    """Index of the hole chosen by the rule, or None if none is big enough."""
    fits = [i for i, (_, h) in enumerate(holes) if h >= size]
    if not fits:
        return None
    if rule == "first":
        return fits[0]                                  # lowest address
    if rule == "best":
        return min(fits, key=lambda i: holes[i][1])     # smallest that fits
    return max(fits, key=lambda i: holes[i][1])         # worst: largest
```

```
$ python3 fit.py

holes (size@start): 150@300, 450@600, 200@1200, 350@1600, 550@2100 | total = 1700 KB
requests: A=220, B=400, C=130, D=430

--- first-fit ---
  A (220 KB) -> hole 450@600, placed at 600-820, leftover 230 KB
  B (400 KB) -> hole 550@2100, placed at 2100-2500, leftover 150 KB
  C (130 KB) -> hole 150@300, placed at 300-430, leftover 20 KB
```

```
  D (430 KB): no hole large enough -> waits (free 950 KB, largest hole 350 KB)
  holes left: 20@430, 230@820, 200@1200, 350@1600, 150@2500 | total free = 950 KB
--- best-fit ---
  A (220 KB) -> hole 350@1600, placed at 1600-1820, leftover 130 KB
  B (400 KB) -> hole 450@600, placed at 600-1000, leftover 50 KB
  C (130 KB) -> hole 130@1820, placed at 1820-1950, leftover 0 KB
  D (430 KB) -> hole 550@2100, placed at 2100-2530, leftover 120 KB
  holes left: 150@300, 50@1000, 200@1200, 120@2530 | total free = 520 KB
--- worst-fit ---
  A (220 KB) -> hole 550@2100, placed at 2100-2320, leftover 330 KB
  B (400 KB) -> hole 450@600, placed at 600-1000, leftover 50 KB
  C (130 KB) -> hole 350@1600, placed at 1600-1730, leftover 220 KB
  D (430 KB): no hole large enough -> waits (free 950 KB, largest hole 330 KB)
  holes left: 150@300, 50@1000, 200@1200, 220@1730, 330@2320 | total free = 950 KB
```

Tek bir örnek genel olarak hiçbir şey kanıtlamaz: en iyi uyan, işe yaramayan çok sayıda küçük delik bırakma eğilimindedir (burada 50 KB; başka dizilerde birkaç baytlık delikler). Silberschatz'ın aktardığı benzetimler, ilk uyan ve en iyi uyanın hem zaman hem de bellek kullanımı açısından en kötü uyandan daha iyi olduğunu gösterir; bellek kullanımında ilk uyan ile en iyi uyandan biri açıkça üstün değildir, ama ilk uyan genellikle daha hızlıdır.

4.3 Parçalanma

Dış parçalanma (external fragmentation). Bir isteği karşılamaya yetecek kadar toplam bellek vardır, ama bitişik değildir: birçok küçük deliğe bölünmüştür. İlk uyan çalıştırmasında 950 KB boştu, ama D (430 KB) yüklenemedi. Dış parçalanma, değişken boylu bloklar dağıtan her yöntemin (değişken bölümler, bölütleme) tipik sorunudur.

%50 kuralı

İlk uyanın istatistiksel çözümlemesi (Knuth'a dayanarak Silberschatz'ın aktardığı biçimiyle), bazı iyileştirmelerle bile, N blok tahsis edildiğinde $0.5N$ blok daha parçalanmaya gittiğini gösterir. Bu durumda belleğin kullanılamayabilecek kısmı

$$\frac{0.5N}{N + 0.5N} = \frac{1}{3}$$

olur.

İç parçalanma (internal fragmentation). Bir sürece tahsis edilmiş ama onun kullanmadığı bellek: bir bloğun *içindeki* israf. Bellek sabit boyutlu birimlerle dağıtıldığında ortaya çıkar. $18\,464$ baytlık bir delik bulunduğunu ve bir sürecin $18\,462$ bayt istediğini varsayalım. Listede 2 baytlık bir delik tutmak (liste girdisi) deliğin kendisinden daha pahalıya gelir; bu yüzden tahsis edici $18\,464$ baytın hepsini verir ve 2 bayt iç parçalanmaya gider. İç parçalanma, sabit bölümlerin ve sayfalamanın tipik sorunudur.

4.4 Sıkıştırma

Dış parçalanmanın bir çaresi **sıkıştırmadır** (compaction): bellek içeriği, tüm boş bellek tek bir büyük blokta toplanacak biçimde yeniden düzenlenir. İlk uyan sonucumuzda İS, C, P1, A, P2, P3, P4, B, P5 blokları 1850 KB

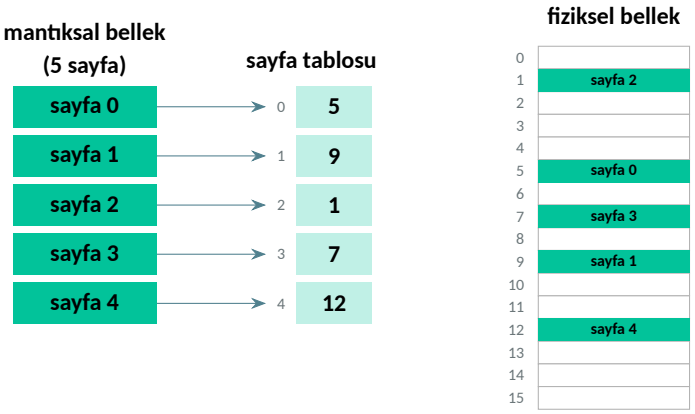
kaplar; belleğin alt ucuna taşındıklarında $2800-1850 = 950$ KB'lık tek bir delik kalır ve D sığar (geriye 520 KB kalır).

Sıkıştırma yalnızca yer değiştirme dinamikse (yürütme zamanında bağlama) mümkündür: işletim sistemi baytları taşır ve taban yazmacını günceller. Üstelik pahalıdır: en kötü durumda belleğin tamamı kopyalanır, ilgili süreçler durdurulmalıdır ve bir aygıt bir tampona yazarken o tampona G/Ç yapan süreç taşınamaz. Daha iyi bir çare, bir sürecin belleğinin *bitişik olmamasına* izin vermektir. Sayfalamanın fikri budur.

5 Sayfalama

5.1 Sayfalar ve çerçeveler

Sayfalama (paging) dış parçalanmayı tamamen ortadan kaldırır. Fiziksel bellek **çerçeve** (frame) adı verilen sabit boyutlu bloklara, mantıksal bellek ise **sayfa** (page) adı verilen aynı boyda bloklara bölünür. Bir süreç yüklendiğinde sayfaları *herhangi* boş çerçevelere yerleştirilir. Süreç başına bir **sayfa tablosu** (page table) her sayfanın hangi çerçevede durduğunu kaydeder. İşletim sistemi bir boş çerçeve listesi tutar.



Şekil 6: Sayfalama: bir sürecin beş sayfası on altı çerçeveye dağılmış. Bu sayfa tablosu aşağıdaki tüm örneklerde ve `page_split.c`'de kullanılmaktadır.

Sayfa boyunu donanım belirler ve her zaman 2'nin bir kuvvetidir; tipik olarak 4 KB (x86-64 üzerinde Linux; `getconf PAGESIZE` 4096 yazdırır), bazen 16 KB (Apple silicon) ya da daha büyük.

5.2 Adresi sayfa numarası ve ofsete bölmek

Mantıksal adres uzayı 2^m bayt (m bitlik adresler) ve sayfa boyu 2^n bayt ise, bir mantıksal adresin üstteki $m-n$ biti **sayfa numarasını** (page number) p, alttaki n biti de **sayfa ofsetini** (page offset, sayfa içi konum) d verir:



$$p = \left\lfloor \frac{a}{2^n} \right\rfloor = a \gg n, \qquad d = a \bmod 2^n = a \,\&\, (2^n - 1), \qquad a_{\text{fiz}} = f \cdot 2^n + d,$$

burada f, sayfa tablosunun p numaralı girdisinde bulunan çerçeve numarasıdır. Sayfa boyu 2'nin kuvveti olduğundan bölme gerekmez: donanım üstteki bitleri doğrudan sayfa tablosunda dizin olarak kullanır, alttaki bitleri değiştirmeden kopyalar. Örneğin 16 bitlik adresler ve 4 KB'lık sayfalarla ($n = 12$) p 4 bittir ve adres uzayında $2^4 = 16$ sayfa vardır; 32 bitlik adreslerle p 20 bittir (2^{20} sayfa).

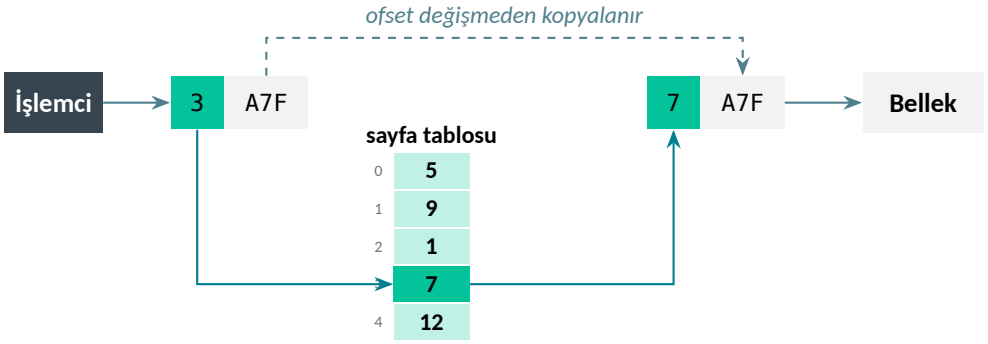
Kullanışlı bir kısayol: bir onaltılık basamak tam olarak 4 bittir; bu yüzden 4 KB'lık sayfalarda ofset, adresin son üç onaltılık basamağıdır.

Mantıksal adres `0x3A7F`'nin çevirisi

16 bitlik adresler, 4 KB'lık sayfalar, yukarıdaki şeklin sayfa tablosu (sayfa $0 \to 5$, $1 \to 9$, $2 \to 1$, $3 \to 7$, $4 \to 12$).
İkilik tabanda `0x3A7F` $= \underbrace{0011}_{p}\ \underbrace{1010\,0111\,1111}_{d}$; dolayısıyla $p = 3$ ve $d =$ `0xA7F`.
Onluk tabanda: `0x3A7F` $= 14975$; $p = \lfloor 14975/4096 \rfloor = 3$, $d = 14975 - 3 \cdot 4096 = 2687 =$ `0xA7F`.
Sayfa tablosu $f = 7$ çerçevesini verir; buradan

$$a_{\text{fiz}} = 7 \times 4096 + 2687 = 28672 + 2687 = 31359 = \texttt{0x7A7F}.$$

Onaltılık tabanda sayfa basamağı 3'ü çerçeve basamağı 7 ile değiştirmek yeterlidir. Benzer biçimde `0x1234` için $p = 1$, $d =$ `0x234`, çerçeve 9, fiziksel `0x9234` $= 37428$; `0x5000` için ise $p = 5$'tir ve bu sayfa sürecin parçası değildir: tuzak.



Şekil 7: Sayfalama donanımı `0x3A7F`'yi `0x7A7F`'ye çeviriyor.

5.3 Sayfalamada iç parçalanma

Sayfalamada dış parçalanma yoktur: her boş çerçeve her sayfayı tutabilir. Ancak bellek tam çerçeveler hâlinde verildiğinden bir sürecin son çerçevesi genellikle dolu değildir.

10.000 baytlık bir süreç

4096 baytlık sayfalarla: $10\,000 = 2 \times 4096 + 1808$. Sürecin 3 çerçeveye ihtiyacı vardır; üçüncüsü yalnızca 1808 bayt tutar, dolayısıyla $4096 - 1808 = 2288$ bayt iç parçalanmaya gider.

En kötü durumda bir süreç n sayfa artı bir bayt ister ve neredeyse bütün bir çerçeveyi boşa harcar. Süreç boyları sayfa boyundan bağımsızsa son sayfa ortalama yarı doludur: beklenen iç parçalanma **süreç başına yarım sayfadır**. Bu, küçük sayfalardan yana bir argümandır; ama daha küçük sayfa daha çok sayfa, daha büyük sayfa tabloları ve daha az verimli disk G/Ç'si demektir. Sayfa boyları zamanla büyüdü. x86-64 üzerinde Linux normalde 4 KB'lık sayfalar kullanır; bellek gereksinimi çok büyük programlar için ayrıca 2 MB ya da 1 GB'lık *dev sayfalar* (huge pages) vardır (test makinemiz `/proc/meminfo` içinde `Hugepagesize: 2048` kB bildiriyor).

5.4 Sayfa tablosu ne kadar büyüktür?

Bir sayfa tablosunda mantıksal adres uzayının her sayfası için bir girdi bulunur.

32 bitlik bir adres uzayı için sayfa tablosu boyutu

32 bitlik mantıksal adresler, 4 KB'lık sayfalar, 4 baytlık sayfa tablosu girdileri:

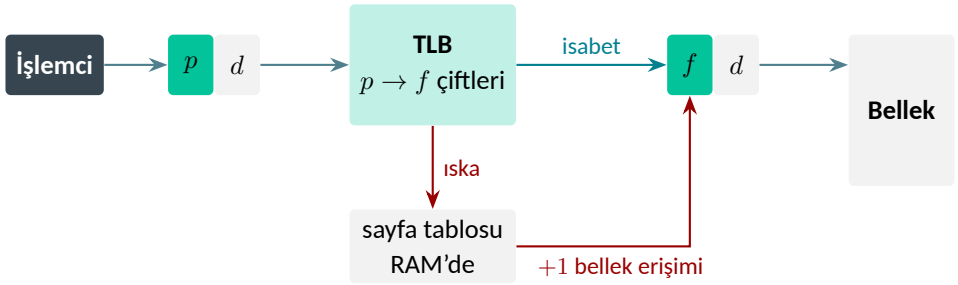
$$\text{girdi sayısı} = \frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576, \qquad \text{boyut} = 2^{20} \times 4\ \text{B} = 2^{22}\ \text{B} = 4\ \text{MB}$$

süreç başına. 100 süreçle bu, çoğu adres uzayının kullanılmayan kısımlarını tanımlayan 400 MB'lık sayfa tablosu demektir. 64 bitlik sistemlerde sorun çok daha büyüktür: 48 bitlik bir adres uzayı için 8 baytlık girdilerle düz bir tablo $2^{36} \times 8$ B $= 512$ GB gerektirirdi.

Burada yalnızca ana hatlarını verdiğimiz çözüm **çok düzeyli** (hiyerarşik, multi-level) sayfa tablosudur: sayfa tablosunun kendisi de sayfalanır. 32 bitlik adresler için iki düzeyli bir yöntemde 20 bitlik sayfa numarası, 2^{10} girdilik bir dış tabloda (tam olarak bir 4 KB'lık sayfa) dizin olan p_1 (10 bit) ile bir iç tabloda dizin olan p_2'ye (10 bit) bölünür; iç tablolar yalnızca adres uzayının gerçekten kullanılan kısımları için var olur. x86-64 dört düzey kullanır ($9+9+9+9$ bit sayfa numarası artı 12 bitlik ofset $=$ 48 bit); yeni işlemciler beşinci bir düzeyi destekler (57 bitlik adresler). Her düzeydeki her tablo tam olarak bir 4 KB'lık sayfaya sığar (8 baytlık 2^9 girdi).

5.5 Çeviri önbelleği (TLB)

Sayfa tablosu ana bellekte tutulur ve bir yazmaç (x86'da CR3) onu gösterir. Bu durumda her veri erişimi *iki* bellek erişimi gerektirir: biri sayfa tablosu girdisi, biri verinin kendisi için. Sayfalama belleğin hızını yarıya indirirdi. Standart çözüm, MMU'nun içinde küçük ve hızlı bir donanım önbelleğidir: **çeviri önbelleği** (translation look-aside buffer, TLB). Her TLB girdisi bir (sayfa numarası, çerçeve numarası) çifti tutar; tüm girdiler paralel olarak aranır (ilişkisel bellek, associative memory). TLB'lerde tipik olarak 32 ile yaklaşık 1536 arasında girdi bulunur.



Şekil 8: TLB'li sayfalama donanımı.

Bir *TLB isabetinde* (hit) çerçeve numarası neredeyse anında elde edilir. Bir *TLB ıskasında* (miss) sayfa tablosu girdisi bellekten okunmalıdır (x86'da donanım, bazı başka mimarilerde işletim sistemi tarafından); ardından girdi, belki daha eski bir girdinin yerine, TLB'ye yazılır. Programlar *yerellik* (locality) gösterdiği için (döngüler aynı kodu yürütür, diziler sırayla gezilir, yığıt hep tepesine yakın kullanılır) %99'un üzerindeki isabet oranları yaygındır. Bağlam değişiminde eski sürecin TLB girdileri boşaltılmalı ya da yeni sürecin kullanamaması için bir adres uzayı kimliğiyle (address-space identifier, ASID) etiketlenmelidir.

Etkin erişim süresi

α **isabet oranı** (hit ratio; sayfa numarasını TLB'de bulan erişimlerin oranı), t_m bir bellek erişiminin süresi ve t_{TLB} bir TLB aramasının süresi olsun.

Bu derste kullanılan uzlaşım

TLB aramasını her erişimde sayarız ve bir ıska fazladan bir bellek erişimine mal olur:

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m.$$

Etkin erişim süresi

$t_{\text{TLB}} = 20$ ns ve $t_m = 100$ ns alalım.

$$\begin{aligned} \alpha = 0.80: &\quad \text{EAT} = 0.80 \times 120 + 0.20 \times 220 = 96 + 44 = 140 \text{ ns} && (t_m\text{'den \%40 yavaş}),\\ \alpha = 0.98: &\quad \text{EAT} = 0.98 \times 120 + 0.02 \times 220 = 117.6 + 4.4 = 122 \text{ ns} && (\%22 \text{ yavaş}). \end{aligned}$$

Hiç TLB olmasaydı her erişim $2t_m = 200$ ns sürerdi. Kısa formülle kontrol: $20 + (2 - 0.98) \times 100 = 122$ ns.

Modern sadeleştirme. Günümüz işlemcilerinde TLB araması yaklaşık bir saat çevrimi sürer ve başka işlerle örtüşür; bu yüzden Silberschatz'ın 10. baskısı onu yok sayar ($t_{\text{TLB}} \approx 0$):

$$\text{EAT} = \alpha\, t_m + (1-\alpha)\, 2t_m.$$

$t_m = 100$ ns ile bu, $\alpha = 0.80$ için 120 ns, $\alpha = 0.99$ için 101 ns verir (Silberschatz $t_m = 10$ ns kullanır: 12 ns ve 10.1 ns). Sınavda hangi uzlaşımı kullandığınızı mutlaka belirtin. Bu bölümdeki tüm sayılar `code/verify.py` ile doğrulanmıştır.

5.6 Koruma bitleri ve geçerli--geçersiz bit

Bir sayfa tablosu girdisi bir çerçeve numarasından fazlasını içerir. Her girdide saklanan koruma bitleri, MMU tarafından her erişimde çeviriyle paralel olarak denetlenir:

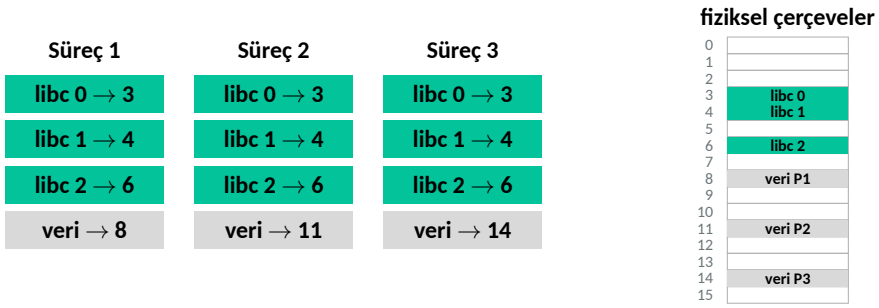
- Bir **geçerli–geçersiz bit** (valid–invalid bit). *Geçerli*, sayfanın sürecin mantıksal adres uzayına ait olduğu; *geçersiz* ise ait olmadığı anlamına gelir ve her erişim işletim sistemine tuzak üretir.
- **Okuma / yazma / yürütme** (read / write / execute) izinleri. Salt okunur bir sayfaya yazmak ya da yürütme yok olarak işaretlenmiş bir sayfadan komut yürütmeye çalışmak (x86-64'ün NX biti) da tuzak üretir.

Sayfa	Çerçeve	Geçerli	İzinler	İçerik
0	5	v	`r-x`	kod
1	9	v	`r-x`	kod
2	1	v	`rw-`	veri
3	7	v	`rw-`	öbek (heap)
4	12	v	`rw-`	yığıt (stack)
5–15	—	i	—	sürecin parçası değil

Tabloda v geçerli, i geçersiz demektir. Bu tabloyla `0x5000` mantıksal adresi (sayfa 5) sayfa geçersiz olduğu için tuzağa düşer; 0. sayfaya yazmak da kod sayfaları salt okunur olduğu için tuzağa düşer. Aynı `r`, `w`, `x` harfleri, `/proc/self/maps` çıktısında da görünür (Bölüm 7). Gelecek hafta geçerli biti ikinci bir işlev kazanacak: bir sayfa sürece ait olup şu anda RAM'de olmayabilir (bir *sayfa hatası*, page fault, 7. hafta).

5.7 Paylaşılan sayfalar

Sayfalama kod paylaşımını kolaylaştırır. Kod **yeniden girilebilirse** (reentrant; kendini hiç değiştirmiyorsa), fiziksel bellekteki tek bir kopyası birçok sürecin sayfa tablosuna eşlenebilir. Tipik bir örnek C kütüphanesi `libc`'dir: bir Linux makinesindeki her süreç onu kullanır, ama kod sayfaları RAM'de yalnızca bir kez bulunur. Yalnızca her sürecin özel veri sayfaları ayrıdır.



Şekil 9: Üç süreç C kütüphanesinin üç kod sayfasını paylaşıyor; yalnızca veri sayfaları özel.

40 süreç ve 2 MB'lık bir kütüphaneyle paylaşım 39×2 MB $= 78$ MB kazandırır. Paylaşılan sayfalar, hiçbir süreç ortak kopyayı değiştiremesin diye salt okunur işaretlenir. Aynı mekanizma `fork()`'un ebeveyn ile çocuk arasında başlangıçta *bütün* sayfaları paylaştırmasını, bir sayfayı ancak biri ona yazdığında kopyalamasını sağlar (*yazınca kopyala*, copy-on-write, 7. hafta).

6 Bölütleme

6.1 Belleğe kullanıcının gözüyle bakış

Programcılar programlarını 4 KB'lık sayfaların bir dizisi olarak düşünmez. Bir ana program, bir dizi fonksiyon, bir yığıt, bir sembol tablosu, global veriler vb. düşünürler. Bunların her biri farklı boyda mantıksal bir birimdir ve her birimin içindeki konumlar, birimin başından uzaklıklarıyla belirtilir ("`sqrt()`'nin beşinci komutu", "yığıtın tepesi"). **Bölütleme** (segmentation) tam olarak bu bakışı destekleyen bir bellek yönetimi yöntemidir:

mantıksal adres uzayı **bölütlerin** (segment) bir topluluğudur ve bir mantıksal adres bir çifttir:

$$\langle \text{bölüt numarası } s,\ \text{ofset } d \rangle.$$

Bölütleri derleyici oluşturur (örneğin kod, global değişkenler, öbek, her iş parçacığı için bir yığıt, standart C kütüphanesi); yükleyici de her birine fiziksel bellekte bir yer atar.

6.2 Bölüt tablosu

Eşleme bir **bölüt tablosu** (segment table) ile yapılır. Her girdide bir **bölüt tabanı** (segment base; bölütün başladığı fiziksel adres) ve bir **bölüt sınırı** (segment limit; bölütün uzunluğu) bulunur. Ofset d, 0 ile sınır arasında olmalıdır; aksi hâlde donanım tuzak üretir:

$$0 \le d < \text{sınır}[s] \text{ ise: } \quad a_{\text{fiz}} = \text{taban}[s] + d; \qquad \text{değilse: tuzak (adresleme hatası).}$$

Başka bir deyişle bölütleme, Bölüm 2'deki taban–sınır fikrinin *her bölüt için* bir yazmaç çiftiyle uygulanmasıdır.

Bölüt tablosuyla çeviri

s	Bölüt	Taban	Sınır	Kapladığı alan
0	ana program	2000	1200	2000–3199
1	`sqrt()` kütüphanesi	6500	300	6500–6799
2	yığıt	4800	600	4800–5399
3	sembol tablosu	3400	900	3400–4299
4	öbek / veri	5600	700	5600–6299

- $\langle 2, 53 \rangle$: $53 < 600$, dolayısıyla $a_{\text{fiz}} = 4800 + 53 = 4853$.
- $\langle 3, 852 \rangle$: $852 < 900$, dolayısıyla $a_{\text{fiz}} = 3400 + 852 = 4252$.
- $\langle 1, 299 \rangle$: $299 < 300$, dolayısıyla $a_{\text{fiz}} = 6500 + 299 = 6799$; 1. bölütün son baytı.
- $\langle 0, 1222 \rangle$: $1222 \ge 1200$: **tuzak**. Program ana program bölütünün sonunun ötesine erişmeye çalıştı.

Bölütler anlamı olan birimler olduğundan koruma ve paylaşım anlamlı birim başına ayarlanabilir: kod salt okunur ve paylaşılabilir, yığıt ve veri okunur-yazılır ve özel. Bedeli, bölütlerin değişken boylu olması ve her birinin bitişik olarak tahsis edilmesi gerekmesidir: Bölüm 4'teki sorunlar (yerleştirme, dış parçalanma, sıkıştırma) geri gelir.

6.3 Sayfalama ve bölütleme

	Sayfalama	**Bölütleme**
Birim boyu	sabit (ör. 4 KB), donanım belirler	değişken, programın yapısını izler
Programcı görür mü?	hayır	evet (ana program, yığıt, veri, …)
Mantıksal adres	tek sayı, $p \mid d$ olarak bölünür	bir çift $\langle s, d \rangle$
Dış parçalanma	yok	var
İç parçalanma	var, süreç başına ortalama $\frac{1}{2}$ sayfa	yok
Koruma ve paylaşım	sayfa başına	bölüt (anlamlı birim) başına
Tablo girdisi	çerçeve numarası ve bitler	taban, sınır ve bitler

Sayfalardan oluşan bölütler

Birçok tarihsel sistem iki yöntemi birleştirdi: adres uzayı bölütlere, her bölüt de sayfalara bölünüyordu. Multics (1960'lar, UNIX'in atası) böyle çalışıyordu; Intel 80386 (1985) da öyle: 32 bitlik korumalı kipi bölütlenmiş bir adresi önce "doğrusal" (linear) bir adrese çeviriyor, sonra onu sayfalıyordu. Her iki mekanizma da uyumluluk için x86 işlemcilerinde hâlâ bulunur.

6.4 x86-64 Linux gerçekte ne yapar?

64 bit kipte Intel ve AMD işlemcileri bölütlemeyi fiilen devre dışı bırakır: kod, veri ve yığıt bölütlerinin tabanı zorunlu olarak 0'dır ve tüm adres uzayını kapsarlar (*düz*, flat, bellek modeli). Yalnızca FS ve GS bölüt yazmaçları sıfırdan farklı bir taban tutar; Linux bunları iş parçacığına özel verileri bulmak için kullanır. Tüm gerçek çeviri ve koruma işini **sayfalama** yapar:

- 48 bitlik sanal adresler için dört düzeyli sayfa tabloları ($9 + 9 + 9 + 9 + 12$ bit), yeni işlemcilerde beş düzey (57 bit);
- 4 KB, 2 MB ve 1 GB'lık sayfa boyları;
- süreç başına bir sayfa tablosu takımı; CR3 yazmacı en üst düzey tablonun fiziksel adresini tutar ve bir bağlam değişimi (3. hafta) onu yeniden yükler.

ARM64 işlemcilerinde (telefonlar, Apple M serisi) bölütleme hiç yoktur. Bölüt *fikri* yazılımda yaşamaya devam eder: Linux her süreç için bir bellek bölgeleri listesi (program kodu, veri, öbek, yığıt, her paylaşılan kütüphane) tutar ve `/proc/<pid>/maps` tam olarak bunu yazdırır. *Segmentation fault* (bölütleme hatası) iletisi bir ad olarak yaşıyor; ama Linux'ta artık bu bölgelerin dışına bir erişimi ya da sayfa izinlerinin ihlalini bildirir.

7 Uygulama: Linux'ta Gerçek Adresler

Programlar bu notların yanındaki `code/` klasöründedir ve `gcc -Wall -O2` ile (ya da kısaca `make` ile) derlenir. Aşağıdaki tüm çıktılar Linux test makinemizde (x86-64) yakalanmıştır. Linux bölgelerin konumunu her çalıştırmada rastgele değiştirdiği için (**ASLR**, address-space layout randomisation, adres uzayı düzeninin rastgeleleştirilmesi; bir güvenlik önlemi) sizin adresleriniz farklı olacaktır, ama örüntü aynı kalır. Programların yorumları ve

çıktıları İngilizcedir.

7.1 Örnek 1: değişkenlerim nerede duruyor?

Yardımcı fonksiyon `show_maps()` (tam dosya için bkz. `code/mem_layout.c`), `/proc/self/maps` satırlarından program dosyasına, öbeğe ve yığıta ait olanları yazdırır. Programın geri kalanı:

```
int initialised_global = 42;   /* data segment             */
int uninitialised_global;      /* bss segment (zero-filled)  */
```

```
int main(void)
{
    int local = 7;                          /* stack                   */
    int *dynamic = malloc(sizeof(int));     /* heap                    */
    if (dynamic == NULL) return 1;
    *dynamic = 99;

    printf("text  (code)   main()      : %p\n", (void *)main);
    printf("data  (global) initialised : %p\n", (void *)&initialised_global);
    printf("bss   (global) zero-filled : %p\n", (void *)&uninitialised_global);
    printf("heap  (malloc)             : %p\n", (void *)dynamic);
    printf("stack (local variable)     : %p\n", (void *)&local);
    printf("page size                  : %ld bytes\n", sysconf(_SC_PAGESIZE));
    printf("\n--- my regions in /proc/self/maps ---\n");
    show_maps();

    free(dynamic);
    return 0;
}
```

```
$ ./mem_layout
text  (code)   main()      : 0x56412f3601e0
data  (global) initialised : 0x56412f363010
bss   (global) zero-filled : 0x56412f363018
heap  (malloc)             : 0x564160c6c2a0
stack (local variable)     : 0x7ffe905b48c4
page size                  : 4096 bytes

--- my regions in /proc/self/maps ---
56412f35f000-56412f360000 r--p mem_layout
56412f360000-56412f361000 r-xp mem_layout
56412f361000-56412f362000 r--p mem_layout
56412f362000-56412f363000 r--p mem_layout
56412f363000-56412f364000 rw-p mem_layout
564160c6c000-564160c8d000 rw-p [heap]
7ffe90595000-7ffe905bb000 rw-p [stack]
```

Her adresi bir bölgeyle eşleştirin: `main` (`0x…3601e0`) program dosyasının `r-xp` bölgesindedir (okunabilir ve yürütülebilir, yazılamaz: kod); iki global değişken `rw-p` bölgesindedir; `malloc` bloğu `[heap]` içindedir; yerel değişken ise kullanıcı adres uzayının tepesine yakın, `[stack]` içindedir. Her bölge `0x1000` $= 4096$'nın bir katında başlar ve biter: bölgeler tam sayfalardan oluşur. Yığın bölgesi `0x564160c8d000`$-$`0x564160c6c000` $=$

`0x21000` $= 135\,168$ bayt $= 33$ sayfa; yığıt bölgesi `0x26000` $= 155\,648$ bayt $= 38$ sayfadır.

7.2 Kabuktan sayfa boyu ve bellek haritası

```
$ getconf PAGESIZE
4096
$ grep Hugepagesize /proc/meminfo
Hugepagesize:       2048 kB
$ head -n 6 /proc/self/maps
559948881000-559948883000 r--p 00000000 fe:00 270606   /usr/bin/head
559948883000-559948889000 r-xp 00002000 fe:00 270606   /usr/bin/head
559948889000-55994888b000 r--p 00008000 fe:00 270606   /usr/bin/head
55994888b000-55994888c000 r--p 00009000 fe:00 270606   /usr/bin/head
55994888c000-55994888d000 rw-p 0000a000 fe:00 270606   /usr/bin/head
559970df7000-559970e18000 rw-p 00000000 00:00 0        [heap]
```

`/proc/self` her zaman onu okuyan süreci gösterir; burada `head`'in kendisini. Sütunlar şunlardır: sanal başlangıç–bitiş adresi, izinler (`r`, `w`, `x` ve özel için `p` ya da paylaşılan için `s`), dosya içindeki ofset, aygıt, inode ve dosya adı. Örneğin `head`'in kod bölgesi `0x559948889000` − `0x559948883000` = `0x6000` $= 24\,576$ bayt $= 6$ sayfa yer kaplar. (Son sütundaki boşluklar kısaltıldı.)

7.3 Örnek 2: aynı adres, farklı değerler

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int x = 100;                    /* one global variable */

int main(void)
{
    pid_t pid = fork();         /* now there are two processes */

    if (pid == 0) {             /* ---- child ---- */
        x = 200;                /* change OUR copy only        */
        printf("child  (PID %d): &x = %p   x = %d\n",
               getpid(), (void *)&x, x);
        return 0;
    }
    wait(NULL);                 /* ---- parent: wait for the child */
    printf("parent (PID %d): &x = %p   x = %d\n",
           getpid(), (void *)&x, x);
    return 0;
}
```

```
$ ./fork_vaddr
child  (PID 1760): &x = 0x55bc4a29c010   x = 200
parent (PID 1759): &x = 0x55bc4a29c010   x = 100
```

`fork()`'tan sonra çocuk ebeveynin bir kopyasıdır; bu yüzden ikisinin düzeni aynıdır ve &x ikisinde de aynı

sayıdır. Yine de çocuk x'i 200 yaptı ve ebeveyn hâlâ 100 görüyor. &x fiziksel bir adres olsaydı bu imkânsız olurdu: RAM'in tek bir baytı iki farklı değer tutamaz. Adres *mantıksaldır*; her sürecin kendi sayfa tablosu vardır ve bu tablo x'i içeren sayfayı farklı bir fiziksel çerçeveye eşler. (Daha kesin söylersek: `fork()`'tan hemen sonra iki sayfa tablosu da salt okunur işaretlenmiş aynı çerçeveyi gösterir; çocuk yazdığında çekirdek sayfayı kopyalar ve çocuğun girdisini günceller. Bu yazınca kopyala tekniği 7. haftanın konusudur.) Programı birkaç kez çalıştırın: ASLR yüzünden adres her çalıştırmada değişir, ama ebeveyn ve çocuk her zaman aynı adresi görür.

7.4 Örnek 3: bit işlemleriyle sayfa numarası ve ofset

```c
#define N_BITS    12                    /* page size 2^12 = 4096 bytes */
#define PAGE_SIZE (1UL << N_BITS)
#define OFF_MASK  (PAGE_SIZE - 1)       /* 0xFFF: low 12 bits set      */

/* page table of a 16-page (64 KB) address space; -1 = invalid page */
static const int page_table[16] = { 5, 9, 1, 7, 12, -1, -1, -1,
                                    -1, -1, -1, -1, -1, -1, -1, -1 };

int main(int argc, char *argv[])
{
    printf("page size = %lu bytes, offset mask = 0x%lX\n\n",
           PAGE_SIZE, OFF_MASK);
    for (int i = 1; i < argc; i++) {
        unsigned long a = strtoul(argv[i], NULL, 0); /* accepts 0x.. */
        unsigned long p = a >> N_BITS;               /* high bits    */
        unsigned long d = a & OFF_MASK;              /* low 12 bits  */
        printf("0x%04lX (%5lu): p = %2lu, d = 0x%03lX (%4lu)", a, a, p, d, d);
        if (p >= 16 || page_table[p] < 0) {
            printf(" -> invalid: trap to OS\n");
            continue;
        }
        unsigned long f = (unsigned long)page_table[p];
        unsigned long phys = (f << N_BITS) | d;
        printf(" -> frame %2lu -> 0x%04lX (%5lu)\n", f, phys, phys);
    }
    return 0;
}
```

```
$ ./page_split 0x3A7F 0x1234 0x5000 0x0000
page size = 4096 bytes, offset mask = 0xFFF

0x3A7F (14975): p =  3, d = 0xA7F (2687) -> frame  7 -> 0x7A7F (31359)
0x1234 ( 4660): p =  1, d = 0x234 ( 564) -> frame  9 -> 0x9234 (37428)
0x5000 (20480): p =  5, d = 0x000 (   0) -> invalid: trap to OS
0x0000 (    0): p =  0, d = 0x000 (   0) -> frame  5 -> 0x5000 (20480)
```

Program, MMU'nun donanımda yaptığını yazılımda yapar: p'yi elde etmek için 12 bit sağa kaydırır, d'yi elde etmek için `0xFFF` ile VE işlemi uygular, çerçeveyi tabloda bulur ve `(f << 12) | d` ile birleştirir. Sonuçlar çözümlü örnekle örtüşür.

8 Özet

Sekiz cümlede 6. hafta

1. İşletim sistemi RAM'i birçok sürece tahsis etmeli, onları birbirinden korumalı ve yer değiştirmeye izin vermelidir; korumayı donanım yapmalıdır.
2. Adres bağlama derleme, yükleme ya da yürütme zamanında yapılabilir; modern sistemler yürütme zamanında bağlar, dolayısıyla mantıksal (sanal) ve fiziksel adresler farklıdır ve aralarındaki çeviriyi MMU yapar.
3. En basit MMU $0 \leq a <$ sınır koşulunu denetler ve bir taban (yer değiştirme) yazmacı ekler; denetim başarısız olursa tuzak üretilir.
4. Bütün süreçlerin takası saniyeler sürer (50 MB/s hızla 100 MB: her yön için 2 s); bu yüzden modern sistemler bunun yerine sayfaları takas eder.
5. İlk, en iyi ya da en kötü uyanla bitişik tahsis dış parçalanmadan muzdariptir (%50 kuralı: belleğin üçte birine kadarı kaybolabilir); sıkıştırma yardımcı olur ama pahalıdır.
6. Sayfalama belleği eşit sayfalara ve çerçevelere böler; bir adres p (üst bitler) ve d (alt bitler) olarak bölünür; dış parçalanma yoktur, yalnızca süreç başına ortalama yarım sayfalık iç parçalanma vardır.
7. Sayfa tabloları büyüktür (32 bitlik adreslerde süreç başına 4 MB), bu yüzden çok düzeyli tablolar kullanılır; TLB çevirileri önbellekte tutar ve bizim uzlaşımımızla EAT $= t_{\text{TLB}} + (2 - \alpha)\, t_m$'dir.
8. Bölütleme $\langle s, d \rangle$ adresini bölüt başına bir taban ve sınırla çevirir; x86-64 Linux düz bölütler kullanır ve tüm işi sayfalamayla yapar.

9 Sözlük

Adres bağlama (address binding)
Sembolik adları adreslere eşlemek; derleme, yükleme ya da yürütme zamanında yapılır.

Bellek yönetim birimi (MMU)
Mantıksal adresleri fiziksel adreslere çeviren ve korumayı denetleyen donanım.

Bölüt (segment)
Bir programın değişken boylu mantıksal birimi (kod, yığıt, veri, …).

Bölüt tablosu (segment table)
Her bölüt için bir (taban, sınır) çiftinden oluşan tablo.

Çerçeve (frame)
Fiziksel belleğin sabit boyutlu bir bloğu.

Delik (hole)
Tahsis edilmiş bloklar arasındaki boş bellek bloğu.

Dış parçalanma (external fragmentation)
Boş belleğin, her biri kullanılamayacak kadar küçük deliklere bölünmüş olması.

Etkin erişim süresi (effective access time, EAT)
TLB isabetleri ve ıskaları hesaba katılarak bir bellek erişiminin ortalama süresi.

Fiziksel adres (physical address)
Bellek donanımının gördüğü adres.

Geçerli–geçersiz bit (valid–invalid bit)
Bir sayfanın sürecin adres uzayına ait olup olmadığını gösteren sayfa tablosu biti.

İç parçalanma (internal fragmentation)
Bir bloğun içinde tahsis edilmiş ama kullanılmayan bellek (ör. bir sürecin son sayfasının sonu).

İlk uyan / en iyi uyan / en kötü uyan (first-fit / best-fit / worst-fit)
Yeterince büyük ilk / en küçük / en büyük deliği tahsis etmek.

Mantıksal (sanal) adres (logical / virtual address)
İşlemcinin ürettiği ve programın gördüğü adres.

Sayfa (page)
Mantıksal belleğin, bir çerçeveyle aynı boydaki sabit boyutlu bloğu.

Sayfa tablosu (page table)
Sayfa numaralarını çerçeve numaralarına eşleyen, geçerli ve koruma bitleri de taşıyan süreç başına tablo.

Sınır yazmacı (limit register)
Sürecin mantıksal adres uzayının boyunu tutar; sınıra eşit ya da büyük adresler tuzak üretir.

Sıkıştırma (compaction)
Tahsis edilmiş blokları bir araya taşıyarak boş belleği tek bir blokta toplamak.

Taban (yer değiştirme) yazmacı (base / relocation register)
Sürecin fiziksel başlangıç adresini tutar; her mantıksal adrese eklenir.

Takas (swapping)
Bir süreci (ya da bugün, bir sayfayı) ana bellek ile yardımcı bellek arasında taşımak.

TLB (translation look-aside buffer, çeviri önbelleği)
Sayfa tablosu girdilerini tutan küçük, hızlı, ilişkisel önbellek.

Yardımcı bellek (backing store)
Takas edilen belleği tutan hızlı ikincil depolama.

10 Kendiniz Deneyin

Kendi makinenizde belleği keşfedin

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine):

```
$ getconf PAGESIZE            # bayt olarak sayfa boyu
$ cat /proc/self/maps         # cat'in kendi bellek haritası
$ make && ./mem_layout && ./mem_layout   # iki kez çalıştırın: ASLR
$ free -h                     # kullanılan RAM ve takas alanı
$ cat /proc/swaps             # takas alanları
$ vmstat 1 5                  # si/so = saniyede içeri/dışarı takas
```

Windows: Görev Yöneticisi → *Performans* → *Bellek* ekranı "Kullanımda", "Kaydedildi" ve "Disk belleğine alınan havuz" değerlerini gösterir (adlar sürüme göre biraz değişebilir); `systeminfo` komutu disk belleği dosyasının (Windows'un takas alanı) boyunu yazdırır.

1. Makinenizde sayfa boyu ne kadar? (Apple silicon işlemcili bir Mac'te `getconf PAGESIZE` 16384 yazdırır.)
2. `cat`'in `[stack]` bölgesi kaç sayfa büyüklüğünde?
3. `mem_layout`'un yazdırdığı adresler çalıştırmalar arasında değişiyor mu? Bu, saldırıları neden zorlaştırır?
4. Makineniz şu anda takas alanı kullanıyor mu? Ne kadar?

11 Tekrar Soruları

Kavramsal

1. Adres bağlamanın yapılabileceği üç zamanı sayın. Hangisi özel donanım gerektirir ve modern sistemler neden onu kullanır?
2. Mantıksal adres ile fiziksel adres arasındaki fark nedir? Hangisini kim görür?
3. Taban ve sınır yazmaçlarının süreçleri birbirinden nasıl koruduğunu açıklayın. Bu yazmaçları yüklemek neden ayrıcalıklı bir komut olmalıdır?
4. Modern işletim sistemleri neden bütün süreçleri takas etmez?
5. İlk uyan, en iyi uyan ve en kötü uyanı açıklayın. Hangisi genellikle en hızlıdır? Hangisi çok küçük delikler bırakma eğilimindedir?
6. Dış ve iç parçalanmayı tanımlayın ve her birine bir örnek verin. Sayfalama hangisini ortadan kaldırır?
7. %50 kuralını ifade edin. Belleğin ne kadarı kullanılamaz hâle gelebilir?
8. Sayfa boyu neden 2'nin bir kuvveti olmalıdır?
9. TLB hangi sorunu çözer? TLB isabet oranları neden genellikle bu kadar yüksektir?
10. Geçerli–geçersiz bit ne için kullanılır? Tuzağa düşecek bir erişim örneği verin.
11. Kod sayfaları neden süreçler arasında paylaşılabilir de veri sayfaları normalde paylaşılamaz?
12. Sayfalama ile bölütlemeyi üç açıdan karşılaştırın. x86-64 Linux hangisini kullanır?
13. `fork_vaddr`'da ebeveyn ve çocuk aynı adresi ama farklı değerleri yazdırıyor. Açıklayın.

Sayısal

14. Taban $=$ `0x2000`, sınır $=$ `0x0800`. `0x07FF`, `0x0800` ve `0x0123` mantıksal adreslerini çevirin. Geçerli fiziksel aralığı verin.
15. 250 MB'lık bir süreç dışarı, 150 MB'lık bir başkası içeri takas ediliyor; disk 100 MB/s aktarıyor. Takas ne kadar sürer (gecikmeyi yok sayın)?

16. Delikler (bu sırayla) 100, 500, 200, 300, 600 KB. İstekler: 212, 417, 112, 426 KB. İlk uyan, en iyi uyan ve en kötü uyanı uygulayın. Hangi istekler karşılanamaz?

17. Sayfalar 4 KB, adresler 16 bit, sayfa tablosu $0 \to 5, 1 \to 9, 2 \to 1, 3 \to 7, 4 \to 12$. `0x2ABC` ve `0x4010`'u çevirin. `0x5C21`'i p ve d olarak bölün.

18. Bir sürecin $72\,766$ bayta ihtiyacı var; sayfalar 2048 bayt. Süreç kaç çerçeve alır ve ne kadar iç parçalanma oluşur?

19. Mantıksal adresler 32 bit, sayfalar 8 KB ve sayfa tablosu girdileri 4 bayt. Düz bir sayfa tablosunda kaç girdi vardır ve tablo ne kadar büyüktür?

20. $t_{\text{TLB}} = 20$ ns, $t_m = 100$ ns, $\alpha = 0.90$. EAT'yi bizim uzlaşımımızla ve $t_{\text{TLB}} = 0$ sadeleştirmesiyle hesaplayın. 125 ns'lik bir EAT için (bizim uzlaşımımızla) hangi isabet oranı gerekir?

21. Bölüm 6.2'teki bölüt tablosunu kullanarak $\langle 4, 699\rangle$, $\langle 4, 700\rangle$ ve $\langle 3, 0\rangle$ adreslerini çevirin.

Sayısal soruların yanıtları

14. `0x07FF` $<$ `0x0800` $\Rightarrow$ `0x27FF`; `0x0800` sınırdan küçük değil $\Rightarrow$ tuzak; `0x0123` $\Rightarrow$ `0x2123`. Geçerli fiziksel aralık `0x2000–0x27FF`.

15. $250/100 + 150/100 = 2.5 + 1.5 = 4$ s.

16. İlk uyan: $212 \to 500$ (kalan 288), $417 \to 600$ (kalan 183), $112 \to 288$ (kalan 176), 426 beklemek zorunda. En iyi uyan: $212 \to 300$, $417 \to 500$, $112 \to 200$, $426 \to 600$: hepsi sığar. En kötü uyan: $212 \to 600$ (kalan 388), $417 \to 500$, $112 \to 388$, 426 beklemek zorunda.

17. `0x2ABC`: $p = 2$, $d =$ `0xABC`, çerçeve $1 \Rightarrow$ `0x1ABC` $= 6844$. `0x4010`: $p = 4$, $d =$ `0x010`, çerçeve $12 \Rightarrow$ `0xC010` $= 49168$. `0x5C21`: $p = 5$, $d =$ `0xC21` $= 3105$.

18. $72\,766 = 35 \times 2048 + 1086$, dolayısıyla 36 çerçeve; iç parçalanma $2048 - 1086 = 962$ bayt.

19. $2^{32}/2^{13} = 2^{19} = 524\,288$ girdi; $2^{19} \times 4$ B $= 2^{21}$ B $= 2$ MB.

20. Bizim uzlaşımımız: $20 + (2 - 0.9) \times 100 = 130$ ns ($= 0.9 \times 120 + 0.1 \times 220 = 108 + 22$). Sadeleştirilmiş: $0.9 \times 100 + 0.1 \times 200 = 110$ ns. 125 ns için: $20 + (2 - \alpha)\,100 = 125 \Rightarrow \alpha = 0.95$.

21. $\langle 4, 699\rangle$: $699 < 700 \Rightarrow 5600 + 699 = 6299$. $\langle 4, 700\rangle$: $700 \geq 700 \Rightarrow$ tuzak. $\langle 3, 0\rangle$: 3400.

12 İleriye Bakış: 7. Hafta

- Sanal bellek: bir sürecin tamamen RAM'de olması gerekmez; isteğe bağlı sayfalama (demand paging) ve sayfa hataları (page fault).
- `fork()`'tan sonra yazınca kopyala (copy-on-write).
- Sayfa değiştirme algoritmaları: FIFO, en iyi (optimal, OPT) ve LRU; boşa çalışma (thrashing).
- Dosya sistemleri: dosya kavramı, erişim yöntemleri ve dizin yapıları.

7. hafta, 1–7. haftaları kapsayan 8. hafta ara sınavından önceki son derstir.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley, Bölüm 9 (Main Memory).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, ücretsiz çevrim içi bölümler:
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-intro.pdf` (Bölüm 13, adres uzayları)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-mechanism.pdf` (Bölüm 15, taban ve sınır)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-segmentation.pdf` (Bölüm 16, bölütleme)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-freespace.pdf` (Bölüm 17, boş alan yönetimi)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf` (Bölüm 18, sayfalama)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf` (Bölüm 19, TLB'ler)
  - `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-smalltables.pdf` (Bölüm 20, daha küçük sayfa tabloları)